A snapshot of a simulated Bio-Terror attack on a major population center Based on Operation Dark Winter and Tom Clancy's The Division

3546619

***Abstract*—This document is a report detailing:**
The scenario, ER Diagram, What data was gathered and how, SQL Table Creation, SQL queries, Discussion of complexity.

I. Scenario

The scenario covers a snapshot of an infection simulation, based on Operation Dark Winter[1] from 2001, this tabletop exercise used multiple people as representatives of US leadership and used real population census data along with medical facility capacity to see what would happen in a theoretical Biological terrorist attack, using small pox, on a major population center, such as Manhattan for its very dense population per square kilometer. This exercise concluded that public services would be overloaded and incapable of providing aid in a matter of days and any services aiming to preserve social stability (e.g. police and national guard) would quickly lose control over people, due to a combination of personnel deaths, fear, civil unrest, etc.

The scenario we have here aims to do a simpler version of Operation Dark Winter and only give a look at 1 month after initial outbreak rather than a full simulation until the complete loss of civil safety and catastrophic loss of life, moreover this scenario uses a fictional plague known as the dollar flu[2] (it does have many other names, one being green poison, for the sake of simplicity and avoiding confusion it will only be referred to as the dollar flu here) which is a bio engineered weapon, and combines small pox, H1N1, Ebola, anthrax, and other unnamed pathogens, it also is engineered to become immediately infectious unlike other diseases that only become infectious after their incubation period, this giving the dollar flu a mortality rate of 70-80 percent when care is available and 95 percent mortality once care options fail or are no longer available, moreover given its virtually immediate contagion stage, thus giving it a 90 percent infection rate.

II. ER Diagram

The entity relationship diagram shows how the data from Neighbourhoods contains a lot of data from the Blocks table, this being done with the Neighbourhood ID foreign key. The blocks table has many infection events as the Block ID foreign key allows a single block to contain many infection events. And the Infection Events table leads to the Transmission table that contains the Compound key Event Number that combines Block ID And Event Number.

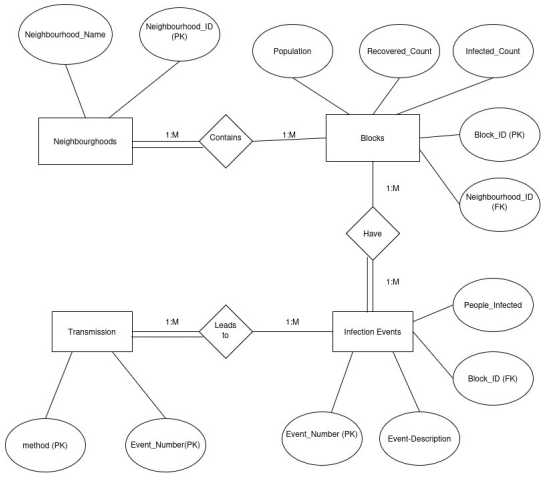


Fig. 1. The entity relationship diagram used to make the connections between tables

III. What Data was gathered and how

Now enough about what the scenario is based on. In terms of data we gathered data from sourced like the official 2020 manhattan census, or tourist information provided by .gov websites[3][4], although it isnt 1:1 with real data as it would have made this scenario mildly uncomfortable, it is quite a sensitive topic to simulate a major casualty event (I'm saying this because there were a lot of warnings from google stating possible safety concerns that mildly slowed down research).

Our data covers the main neighbourhood names and splits their population into blocks, then these blocks are what is also used to represented the number of infected and recovered people, which is also tied to the different infection events setting up a basic timeline along with forms of transmission in each infection event. This is spread over 4 csv tables which are then added to one central SQL database.

The data was cleaned in excel for formatting errors and missing cells in order to maintain a high level of data integrity and give accurate results.

IV. SQL Table Creation

The databse tables were created using CREATE TABLE statements, with primary and foreign keys used to establish connections between entities. The Neighbourhood table was created first to store each neighbourhood's ID and name.

The CityBlock table came next in order to store population, infected cound and recovered count to further expand on each neighbourhood and show population spread, it uses Block ID as its primary key and Neighbourhood ID as its foreign key, ensureing a reference to the Neighourhood Table. The InfectionEvent table records individual infection events, which are there to show how the infection spread in different areas, and it has Event Number as its primary key and Block ID as its foreign key referencing the corresponding block. Ultimately, the Transmission table stores the transmission methods associated with each infection event. It uses a composite key consisting of Event Number and Method, allowing multiple transmission methods to be associated with an event while tables remain consistent. Once the tables have been created, data was imported from CSV files using LOAD DATA INFILE statements, allowing the records to be poppulated efficiently.

As for example insertions the fig. 2. shows:

```
INSERT INTO neighbourhood(Neighbourhood_ID,
Neighbourhood_Name)

VALUES (21, "random_neighbourhood");
```

☐	Edit Copy Delete	19	Chinatown
☐	Edit Copy Delete	20	Financial
☐	Edit Copy Delete	21	random_neighbourhood

Fig. 2. The results of a INSERT operation

Where we use random neighbourhood, as every neighbourhood was already there.

V. SQL Queries

Here I will go over how each SQL query was used and why.

A. Query 1

First query:

```
SELECT * FROM neighbourhood WHERE
Neighbourhood_Name LIKE "%Side";
```

this resulted in showing neighbourhoods with names ending in side, thus showing that the database can be searched effectively. Fig. 3. shows the results of the sql.

←T→		Neighbourhood_ID	Neighbourhood_Name
☐	Edit Copy Delete	2	Upper West Side
☐	Edit Copy Delete	3	Upper East Side
☐	Edit Copy Delete	16	Lower east Side

Fig. 3. Showing the results of using like in the database

B. Query 2

Second query:

```
Event_Description, Infected_Count
FROM infectionevent
INNER JOIN cityblock
ON infectionevent.Block_ID =
cityblock.Block_ID

WHERE People_Infected > 7;
```

This query aims to show the difference between the initial infection spread in a block along with the amount of infected individuals at the end of our snapshot.

Also WHERE is used as a filter, so figures 4 and 5 will show the difference between the command with and without WHERE, and also the results of the query as a whole.

Block_ID	Event_Number	People_Infected	Event_Description	Infected_Count
13	1	10	Infected dollar bills used in a local mall, beginn...	40713
24	3	8	several infected commute on the subway leading to ...	20805
51	4	12	Street market vendors come into contact with infec...	52146
53	6	15	Infected cash passes through a busy ATM area	15528
22	7	20	Airborne particles spread the infection at a theat...	17210

Fig. 4. Shows the result of the above SQL statement with WHERE in it

Block_ID	Event_Number	People_Infected	Event_Description	Infected_Count
13	1	10	Infected dollar bills used in a local mall, beginn...	40713
14	2	5	Infected individuals from the original infection e...	42624
24	3	8	several infected commute on the subway leading to ...	20805
51	4	12	Street market vendors come into contact with infec...	52146
3	5	6	The infection spreads through airborne particles a...	27510
53	6	15	Infected cash passes through a busy ATM area	15528
22	7	20	Airborne particles spread the infection at a theat...	17210
33	8	4	Staff and other patients are exposed to the virus ...	19824
41	9	7	Both infected and uninfected individuals are group...	22621
16	10	5	Infected staff carry the virus into a retirement h...	38560

Fig. 5. Shows the same result without WHERE

C. Query 3

Third query:

```
SELECT Block_ID, Neighbourhood_ID,
Population, Infected_Count

FROM cityblock
WHERE Infected_Count > 40000;
```

This query aims to show the hardest hit blocks, in this case with a infection count over 40000, WHERE being used as a filter, and fig. 6. below shows the results of the query

←T→		Block_ID	Neighbourhood_ID	Population	Infected_Count
☐	Edit Copy Delete	9	2	48769	46331
☐	Edit Copy Delete	10	2	43785	41596
☐	Edit Copy Delete	11	2	46087	43783
☐	Edit Copy Delete	12	2	44789	42550
☐	Edit Copy Delete	13	2	42856	40713
☐	Edit Copy Delete	14	3	44867	42624
☐	Edit Copy Delete	15	3	42890	40746
☐	Edit Copy Delete	17	3	48356	45938
☐	Edit Copy Delete	18	3	46899	44554
☐	Edit Copy Delete	51	19	54890	52146
☐	Edit Copy Delete	52	19	45220	42959

Fig. 6. Shows the results of a query using '¿'

D. Query 4

Fourth query:

```
SELECT Neighbourhood_ID, SUM(Infected_Count),
COUNT(Block_ID)

FROM cityblock
GROUP BY Neighbourhood_ID;
```

The aim of this query is to show the total number of blocks along with total infection amount per neighbourhood rather than per block, and figure 7 shows the results of the SQL querie.

Neighbourhood_ID	SUM(Infected_Count)	COUNT(Block_ID)
1	247026	8
2	214973	5
3	212422	5
4	50795	3
5	17210	1
6	98102	5
7	27876	2
8	2565	1
9	19714	2
10	59696	3
11	17656	1
12	49349	2
13	21229	2
14	65791	3
15	12887	2
16	47811	2
17	8545	1
18	20432	2
19	95105	2
20	34199	2

Fig. 7. Query showing the results of a GROUP BY query

E. Query 5

Fifth query

```
SELECT Block_ID, Neighbourhood_ID,
Population, Infected_Count

FROM cityblock
WHERE Neighbourhood_ID IN (1, 2, 3);
```

This query shows all the blocks and infection counts from neighbourhoods 1,2 and 3, this being further shown in figure 9.

F. Query 6

Sixth query:

```
UPDATE cityblock
SET Recovered_Count =
Recovered_Count - 1 --
de-incrementing the recovered
count for certain rows
WHERE Block_ID IN (1, 15, 51);
```

This query aims to update 3 lines, that were identified to have infectedcount and recoveredcount add up to 1 digit higher than the total population.

	Block_ID	Neighbourhood_ID	Population	Infected_Count
Edit Copy Delete	1	1	32250	30638
Edit Copy Delete	2	1	38543	36616
Edit Copy Delete	3	1	28958	27510
Edit Copy Delete	4	1	35986	34187
Edit Copy Delete	5	1	31435	29863
Edit Copy Delete	6	1	30586	29057
Edit Copy Delete	7	1	27688	26304
Edit Copy Delete	8	1	34580	32851
Edit Copy Delete	9	2	48769	46331
Edit Copy Delete	10	2	43785	41596
Edit Copy Delete	11	2	46087	43783
Edit Copy Delete	12	2	44789	42550
Edit Copy Delete	13	2	42856	40713
Edit Copy Delete	14	3	44867	42624
Edit Copy Delete	15	3	42890	40746
Edit Copy Delete	16	3	40589	38560
Edit Copy Delete	17	3	48356	45938
Edit Copy Delete	18	3	46899	44554

Fig. 8. SQL query using IN

	Block_ID	Neighbourhood_ID	Population	Infected_Count
Edit Copy Delete	1	1	32250	30638
Edit Copy Delete	2	1	38543	36616
Edit Copy Delete	3	1	28958	27510
Edit Copy Delete	4	1	35986	34187
Edit Copy Delete	5	1	31435	29863
Edit Copy Delete	6	1	30586	29057
Edit Copy Delete	7	1	27688	26304
Edit Copy Delete	8	1	34580	32851
Edit Copy Delete	9	2	48769	46331
Edit Copy Delete	10	2	43785	41596
Edit Copy Delete	11	2	46087	43783
Edit Copy Delete	12	2	44789	42550
Edit Copy Delete	13	2	42856	40713
Edit Copy Delete	14	3	44867	42624
Edit Copy Delete	15	3	42890	40746
Edit Copy Delete	16	3	40589	38560
Edit Copy Delete	17	3	48356	45938
Edit Copy Delete	18	3	46899	44554

Fig. 9. Shows a SQL query using UPDATE

VI. Discussion of Complexity

The most difficult part of this scenario would be implementing a fully dynamic infection simulation using just SQL. This means infected and recovered count would change over time, and each change could affect other city blocks, or infection events could start adding and changing the counts of other blocks as tie goes on. Modelling this wold require extensive and repeated calculations along with further variables that change dynamically, examples being healthcarea capacity and availability of emergency services. This would also need more tables to store historical data and maintain relationships between events.

A more practical approach to such an endeavour would be using Python for calculations and SQL for storing and analysing data. As much as SQL supports complex queries, updates and stored procedures, managing the many stages of a ever evolving simulation would be very difficult to maintain. Therefore separating the calculations and data storage/analysis would make the overall maintenance easier, while also avoiding any errors while handling such data.

References

[1] "Dark Winter — Johns Hopkins Center for Health Security," centerforhealthsecurity.org. https://centerforhealthsecurity.org/our-work/tabletop-

exercises/dark-winter-a-training-tabletop-exercise (accessed Oct. 29, 2026).

[2] Contributors to The Division Wiki, “Green Poison,” The Division Wiki. https://thedivision.fandom.com/wiki/Green$_Poison(accessedOct.29, 2026)$.

[3] “Explore Census Data,” data.census.gov, 2020. https://data.census.gov/profile/New$_York?g = 040XX00US36(accessedOct.29, 2026)$.

[4] U. S. C. Bureau, “New York State Population Topped 20 Million In 2020,” Census.gov, Mar. 12, 2025. https://www.census.gov/library/stories/state-by-state/new-york.html (accessed Oct. 29, 2026).